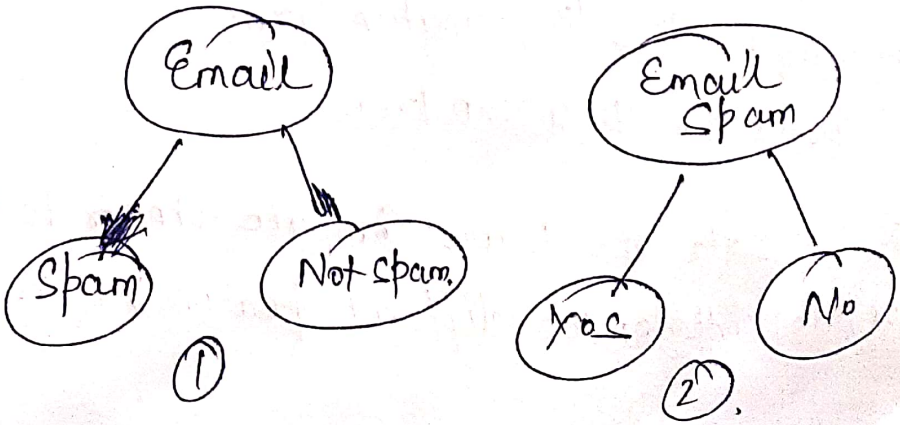
Module II : Supervised Learning : Decision Tree classifier, Naive Bayes classifier, Support Vector Machine (SVM) classifier.
Unsupervised Learning : K-means clustering, Hierarchial clustering
Underfitting Vs Overfitting : Regularization and Bias/Variance.
Ensemble methods : Bagging, Boosting, Improving classification with Adaboost algorithm.

Decision Tree classifier : A decision tree is a supervised machine learning algorithm used for classification tasks. It models decisions in a tree like structure, similar to flowchart, to predict the class label of a given input.

Structure of a Decision Tree :

(a) Root Node : The starting of the tree, representing the initial decision or feature used to split the data.
(b) Internal Nodes : Also known as decision nodes or branches, these represents tests on specific features, leading to different outcomes based on the feature's value.
(c) Branches : Connect internal nodes and represent the possible outcomes or values of the tested feature.
(d) Leaf Nodes : Also known as terminal nodes, these represent the final class labels or predictions after traversing the tree.

Example of Decision Tree



Scanned with OKEN Scanner

Core Concepts: (ID3 algorithm, Iterative Dichotomizer 3)

① **Entropy**: This measures the impurity or disorder within a set of data. The formula for entropy is define as

$$H(S) = -\sum (Pi * \log 2(Pi))$$

where pi is the proportion of instances of class Ci in the set.

② **Information Gain**: This quantifies how much a particular attribute reduces the uncertainity (entropy) in a dataset.

$$\text{Information Gain}(S, A) = \text{Entropy}(S) - \sum [(|Sv| / |S|) * \text{Entropy}(Sv)]$$

S - The Current dataset
A - The attribute being considered for the split

Entropy(S) - The entropy of parent node
Sv - The subset of data in S where attribute A has specific value of
|Sv| - The no. of samples in subset Sv
|S| - The total no. of samples in dataset S.
Entropy (Sv): The entropy of subset Sv.
$\sum$: The summation of over all possible values v of attribute A.

For the Example Given Below

① Compute the Entropy
② Compute the Information Gain
③ Draw the Decision Tree

For the Data Given Below Compute whether to play the football or not, dependency on the different feature values.

Scanned with OKEN Scanner

Day	Weather	Temperature	Humidity	Wind	Play Football
Day1	Sunny	Hot	High	Weak	No
Day 2	Sunny	Hot	High	Strong	No
Day 3	Cloudy	Hot	High	Weak	Yes
Day 4	Rain	Mild	High	Weak	Yes
Day 5	Rain	Cool	Normal	Weak	Yes
Day 6	Rain	Cool	Normal	Strong	No
Day 7	Cloudy	Cool	Normal	Strong	Yes
Day 8	Sunny	Mild	High	Weak	No
Day 9	Sunny	Cool	Normal	Weak	Yes
Day 10	Rain	Mild	Normal	Weak	Yes
Day 11	Sunny	Mild	Normal	Strong	Yes
Day 12	Cloudy	Mild	High	Strong	Yes
Day 13	Cloudy	Hot	Normal	Weak	Yes
Day 14	Rain	Mild	High	Strong	No

Sol: (1) Calculating Information Gain (IG) of Weather.

~~S{+9,-5}~~

Step 1: Entropy of entire dataset

$$S\{+9,-5\} = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.94.$$

Step 2: Entropy of all attributes:

(a) Entropy of Sunny $\{+2,-3\} = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} = 0.97$

(b) Entropy of Cloudy $\{+4,-0\} = -\frac{4}{4}\log_2\frac{4}{4} - \frac{0}{4}\log_2\frac{0}{4} = 0$

(c) Entropy of Rain $\{+3,-2\} = -\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5} = 0.97$

(d) Information Gain = Entropy(whole data) $- \frac{5}{14}$ Ent(S) $- \frac{4}{14}$ Ent(O) $- \frac{5}{14}$ Ent(R)

Information Gain = $= 0.246$

② Calculate IG of Temperature

Step 1: Entropy of Entire dataset

$S(+9,-5) = 0.94$

Step 2: Entropy of all attributes

(a) Entropy of Hot $(+2,-2) = -\frac{2}{4}\log_2\frac{2}{4} - \frac{2}{4}\log_2\frac{2}{4} = 1.0$

(b) Entropy of Mild $\{+4,-2\} = -\frac{4}{6}\log_2\frac{4}{6} - \frac{2}{6}\log_2\frac{2}{6} = 0.91$

(c) Entropy of Cold $(+3,-1) = -\frac{3}{4}\log_2\frac{3}{4} - \frac{1}{3}\log_2\frac{1}{3} = 0.81$

(d) Information Gain = Entropy of whole data $- \frac{4}{14}$ Ent(H) $- \frac{6}{14}$ Ent(m) $- \frac{4}{14}$ Ent(C)

Information Gain = 0.029

③ Calculate IG of Humidity

$S(+9,-5) = 0.94$ (Step 1, the entropy of entire dataset)

Step 2: Entropy of all attributes:

(a) Entropy of High $\{+3,-4\} = -\frac{3}{7}\log_2\frac{3}{7} - \frac{4}{7}\log_2\frac{4}{7} = 0.98$

(b) Entropy of Normal $\{+6,-1\} = -\frac{6}{7}\log_2\frac{6}{7} - \frac{1}{7}\log_2\frac{1}{7} = 0.59$

(c) Information Gain = Entropy (whole data) $- \frac{7}{14}$ Ent(H) $- \frac{7}{14}$ Ent(N)

Information Gain = 0.15

④ Calculate IG of Wind

Step 1: Entropy of entire dataset

$S(+9,-5) = 0.94$

Step 2: Entropy of all attributes:

(a) Entropy of Strong $\{+3,-3\} = -\frac{3}{6}\log_2\frac{3}{6} - \frac{3}{6}\log_2\frac{3}{6} = 1.0$

(b) Entropy of Normal $\{+6,-2\} = -\frac{6}{8}\log_2\frac{6}{8} - \frac{2}{8}\log_2\frac{2}{8} = 0.81$

Scanned with OKEN Scanner

(c) Information Gain = Entropy of whole data $- \frac{6}{14}$ Ent(S) $- \frac{8}{14}$ Ent(W)

Information Gain = 0.0478

∴

① Gain(S, Weather) = 0.246 — Higher

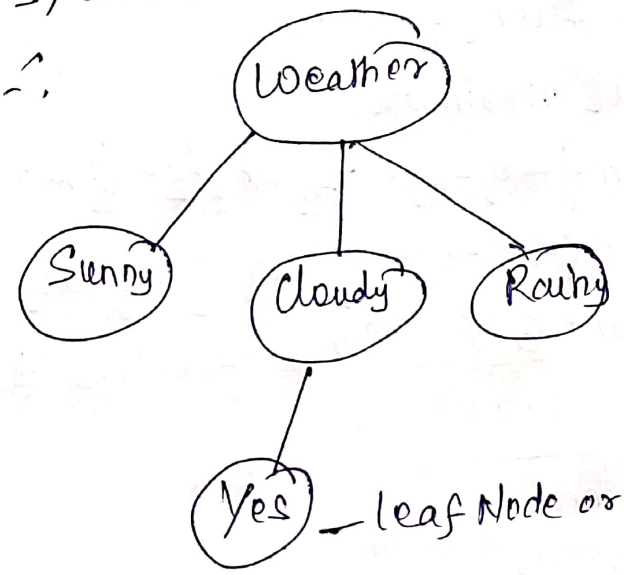
② Gain(S, Temperature) = 0.029.

③ Gain(S, Humidity) = 0.15

④ Gain(S, Wind) = 0.0478

Therefore Weather is Root Node.

∴



Yes — leaf Node or Terminal Node

Taking Sunny data as a separate

Day	Weather	Temp	Humidity	Wind	Play football
Day 1	Sunny	Hot	High	Weak	No
Day 2	Sunny	Hot	High	Strong	No
Day 8	Sunny	Mild	High	Weak	No
Day 9	Sunny	Cool	Normal	Weak	Yes
Day 11	Sunny	Mild.	Normal	Strong	Yes

① Calculate IG of Temperature

Step 1 : Entropy of Sunny (+2, −3) $= -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} = 0.97$

Step 2 : Entropy of all attributes:

Scanned with OKEN Scanner

(a) Entropy of Hot $\{+0,-2\} = -\frac{0}{2}\log_2\frac{0}{2} - \frac{2}{2}\log_2\frac{2}{2} = 0$

(b) Entropy of mild $\{+1,-1\} = \frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} = 1.0$

(c) Entropy of Cool $\{+1,-0\} = -\frac{1}{1}\log_2\frac{1}{1} - \frac{0}{1}\log_2\frac{0}{1} = 0$

(d) Information Gain = Entropy (Sunny) $- \frac{2}{5}$ Ent(H) $- \frac{2}{5}$ Ent(m) $- \frac{1}{5}$ Ent(C)

$= 0.57$

(2) Calculate IG of Humidity

Step 1: Entropy of Sunny $\doteq 0.97$

Step 2: Entropy of all attributes:

(a) Entropy of High $\{+0,-3\} = -\frac{0}{3}\log_2\frac{0}{3} - \frac{3}{3}\log_2\frac{3}{3} = 0$

(b) Entropy of Normal $\{+2,-0\} = -\frac{2}{2}\log_2\frac{2}{2} - \frac{0}{2}\log_2\frac{0}{2} = 0$

(c) Information Gain = Entropy (Sunny) $- \frac{3}{5}$ Ent(H) $- \frac{2}{5}$ Ent(m)

$= 0.97$

(3) Calculate IG of Wind

Step 1: Entropy of Sunny $= 0.97$

Step 2: Entropy of all attributes:

(a) Entropy of Strong $\{+1,-1\} = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} = 1$

(b) Entropy of Weak $\{+1,-2\} = -\frac{1}{3}\log_2\frac{1}{3} - \frac{2}{3}\log_2\frac{2}{3} = 0.918$

(c) Information Gain = Entropy (Sunny) $- \frac{2}{5}$ Ent(S) $- \frac{3}{5}$ Ent(W)

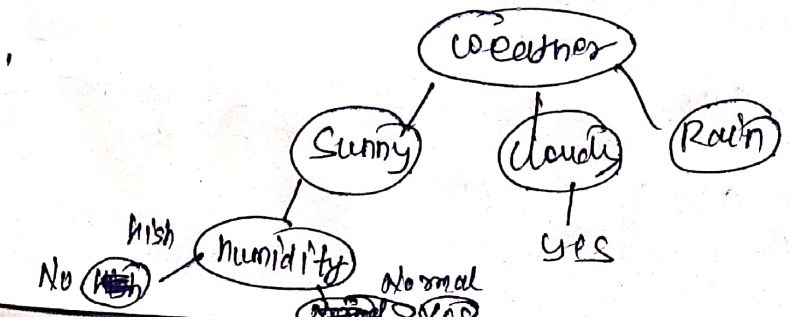
$= 0.019$

$\therefore$ Gain (S_{Sunny}, Temp) $= 0.57$

Gain (S_{Sunny}, humidity) $= 0.97$ — higher

Gain (S_{Sunny}, Wind) $= 0.019$

$\therefore$



Taking Rain Data as separate

Day	Weather	Temperature	Humidity	Wind	Play Football
Day 4	Rain	Mild	High	Weak	Yes
Day 5	Rain	Cool	Normal	Weak	Yes
Day 6	Rain	Cool	Normal	Strong	No
Day 10	Rain	Mild	Normal	Weak	Yes
Day 14	Rain	Mild	High	Strong	No

Calculate IG of Temperature

Step 1: Entropy of Rain $\{+3, -2\} = -\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5} = 0.97$

Step 2: Entropy of all attributes:

(a) Entropy of Hot $\{+0, -0\} = 0$

(b) Entropy of Mild $\{+2, -1\} = -\frac{2}{3}\log\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} = 0.918$

(c) Entropy of Cool $\{+1, -1\} = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} = 1.0$

(d) Information Gain = Entropy(Rain) $- \frac{0}{5}$ Entropy(H) $- \frac{3}{5}$ Ent(m) $- \frac{2}{5}$ (Ent(C)

$= 0.019$

Calculate IG of Humidity

Step 1: Entropy of Rain $\{+3, -2\} = 0.97$

Step 2: Entropy of all attributes:

(a) Entropy of high $\{+1, -1\} = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} = 1$

(b) Entropy of Normal $\{+2, -1\} = -\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} = 0.918$

(c) Information Gain = Entropy(Rain) $- \frac{2}{5}$ Ent(S) $- \frac{3}{5}$ Ent(10)

$= .019$

Calculate IG of Wind

Step 1: Entropy of Rain $\{+3, -2\} = 0.97$

Step 2: Entropy of all attributes

(a) Entropy of Strong $\{+0, -2\} = -\frac{0}{2}\log_2\frac{0}{2} - \frac{2}{2}\log_2\frac{2}{2} = 0$

(b) Entropy of Weak $\{+3, -0\} = -\frac{3}{3}\log_2\frac{3}{3} - \frac{0}{3}\log_2\frac{0}{3} = 0$

(c) Information Gain = Entropy(Rain) $- \frac{2}{5}$ Ent(S) $- \frac{3}{5}$ Ent(W)

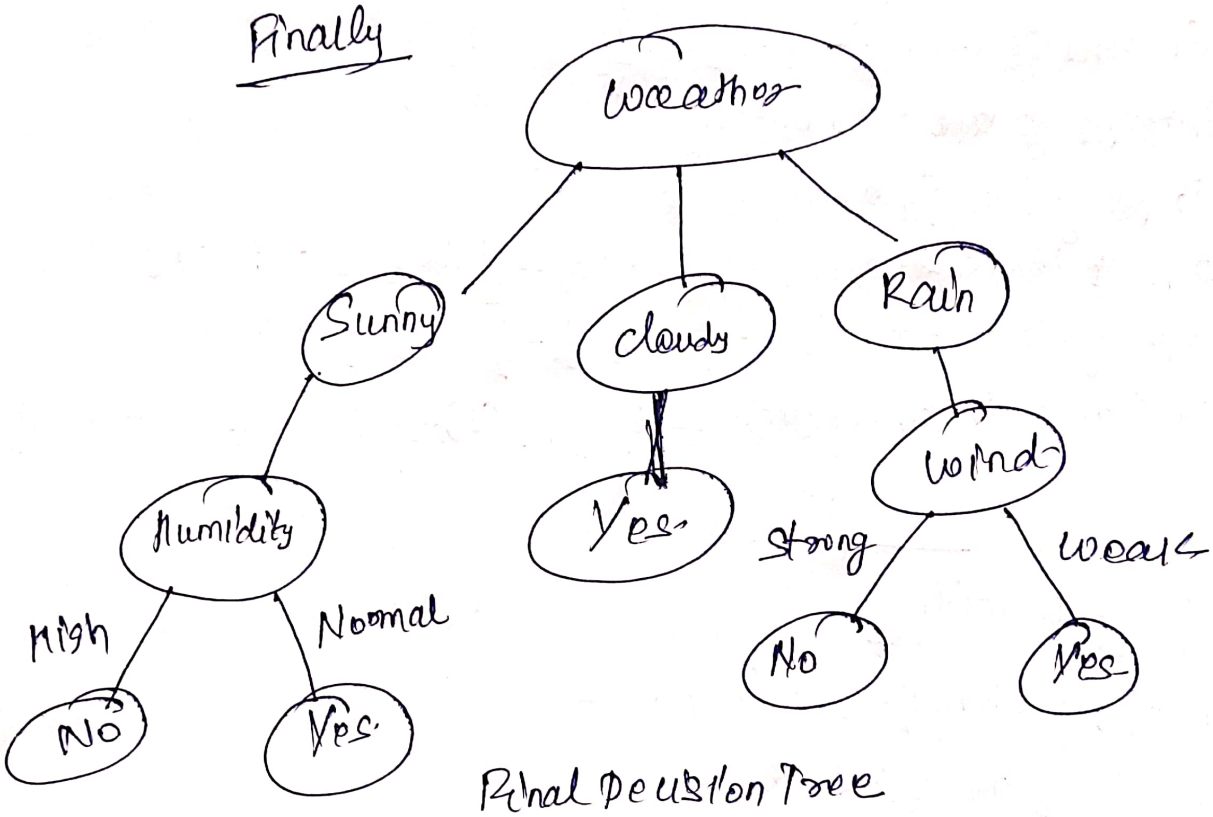
$= 0.97$

$\therefore$ Gain(S Rain, Temp) $= 0.019$

Gain(S Rain, Humidity) $= 0.019$

Gain(S Rain, Wind) $= 0.97$ — Higher

Finally



Final Decision Tree

Naive Bayes classifier:

Bayes Theorem: $P(A|B) = \frac{P(B|A) * P(A)}{P(B)}$ — Given

Basically the Bayes theorem is converted into a machine learning problem.

Change this Bayes Theorem as per the dataset.

Suppose $x = \{x_1, x_2, x_3, x_4 \dots x_n\}$ $\{y\}$

Scanned with OKEN Scanner

$$P(y|x_1,x_2---x_n) = \frac{P(x_1|y)\,P(x_2|y)(P(x_3|y)---P(x_n|y) * P(y)}{P(x_1)\,P(x_2)\,P(x_3)---P(x_n)}$$

Consider an example

Outlook

	Yes	No	P(y)	P(N)
SUNNY	2	3	2/9	3/5
OVERCAST	4	0	4/9	0/5
RAINY	3	2	3/9	2/5
TOTAL	9	5	100%	100%

Temperature

	Yes	NO	P(Y)	P(N)
HOT	2	2	2/9	2/5
MILD	4	2	4/9	2/5
COOL	3	1	3/9	1/5
TOTAL	9	5	100%	100%

PLAY

		P(Y) P(N)
Yes	9	9/14.
No	5	5/14
Total	14	100 %

outlook

Compute : If have Scenario Today is Sunny and Temperature is not ie Today (SUNNY, HOT). Determine whether the person is going to play or not the tennis.

Sol: TODAY(SUNNY, HOT)
(x_1, x_2)

$$\therefore P(\text{Yes}|\text{Today}) = \frac{P(\text{SUNNY}|\text{yes}) \times P(\text{HOT}|\text{yes}) \times P(\text{Yes})}{P(\text{Today})} \Rightarrow \text{skipping}$$

$$= \frac{2}{9} \times \frac{2}{9} \times \frac{9}{14} = \boxed{0.031}$$

$P(\text{No}|\text{Today})$

$$= \frac{3}{5} \times \frac{2}{5} \times \frac{5}{14} = \boxed{0.08571}$$

$P(\text{No}|\text{Today}) = P(\text{SUNNY}|\text{No}) \times P(\text{Hot}|\text{No}) \times P(\text{No})$

Scanned with OKEN Scanner

Normalising the values, by dividing the summation of both.

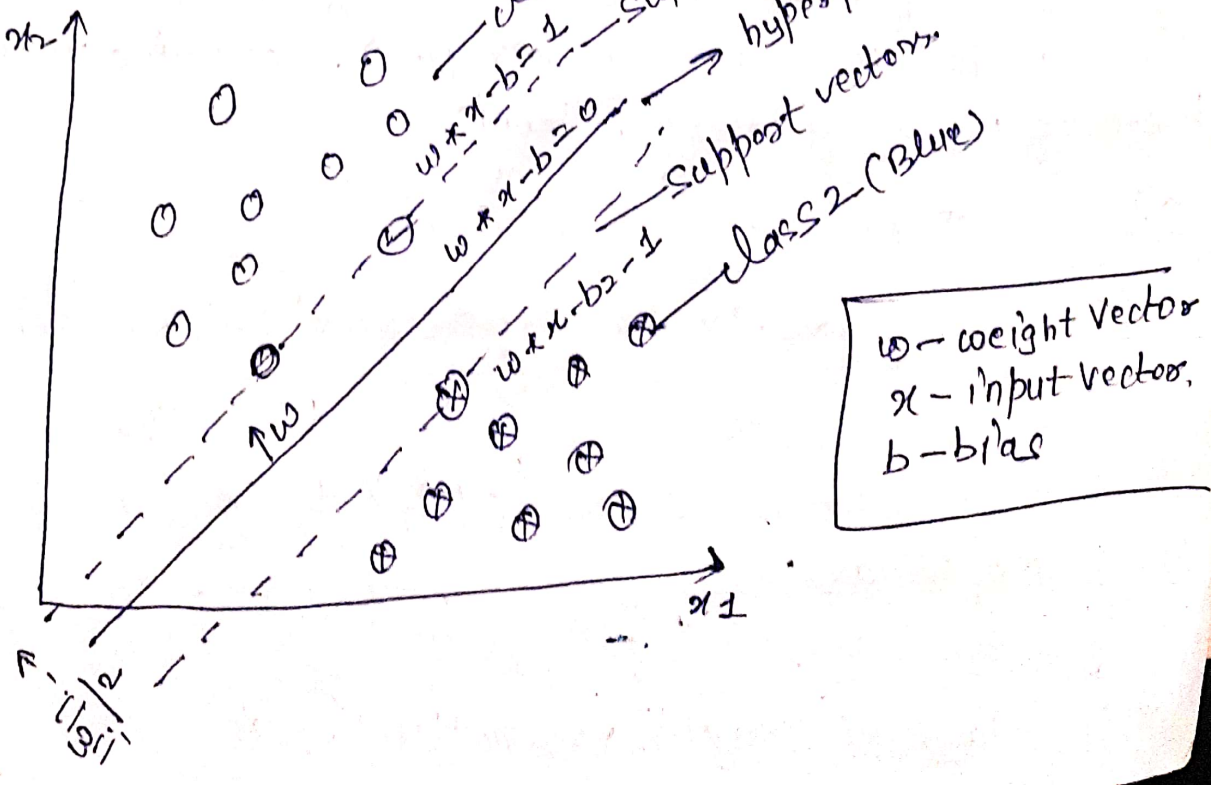
$$\therefore P(Yes) = \frac{0.021}{0.021 + 0.0053} = \frac{0.021}{.1163} = \boxed{0.27}$$

$$\therefore P(No) = 1 - 0.27 = \boxed{.73}$$

Therefore on this particular

Input :- $\boxed{TODAY(SUNNY, NOT) = No}$

Support Vector Machine (SVM) classifier : SVM is a supervised machine learning algorithm commonly used for classification tasks. SVM constructs a hyperplane or a set of hyperplanes in a high dimensional space that separates the different classes. A good separation is achieved by the hyperplane that has the largest margin., meaning the maximum distance between data points of different classes. SVM is a powerful classifier that works both on linearly and nonlinearly separable data. SVM tries to find the "best" margin (distance between the line and the support vectors) that separates the classes.



w - weight vector
x - input vector.
b - bias

(1) Hyperplane: This is the decision boundary represented by the equation $w \cdot x - b = 0$. It separates the two classes (green and blue points). The SVM algorithm searches for the hyperplane that best divides the data while maximizing the margin.

(2) Support Vectors: These are the points closest to the hyperplane from both classes, highlighted on the boundary lines $w \cdot x - b = 1$ and $w \cdot x - b = -1$. In the figure, they are the points on the dashed lines representing the margin. Support vectors directly influence the positioning of the hyperplane

(3) Margin: The margin is the distance between the support vectors and the hyperplane. The goal of the SVM is to maximize the margin, ensuring that the hyperplane separates the classes as clearly as possible. In the figure, the margin is the region between the dashed lines $w \cdot x - b = 1$ and $w \cdot x - b = -1$.

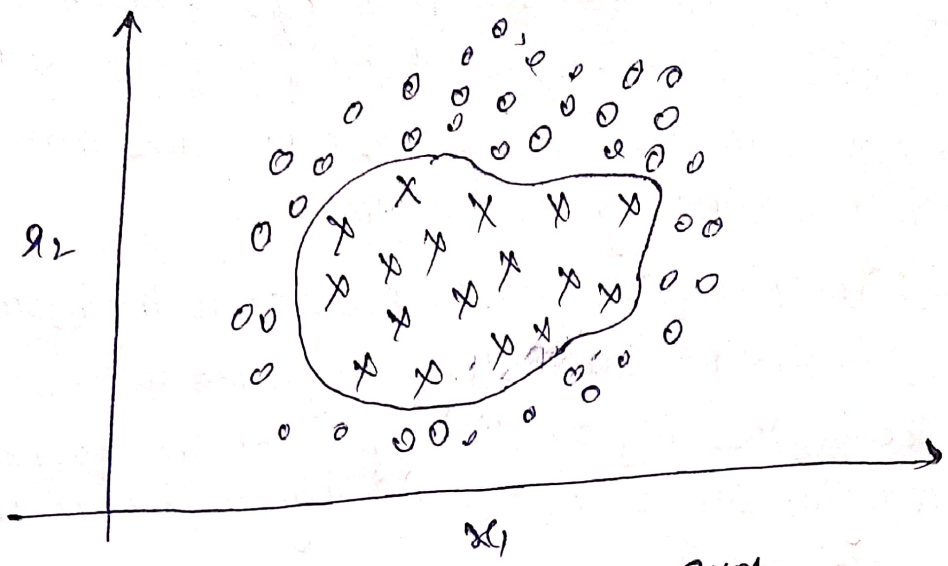
(4) Weight vector w: The arrow labeled w represents the weight vector perpendicular to the hyperplane. The direction of this vector indicates how the hyperplane is oriented, and its magnitude determines how steep the slope of the separation boundary is. Each of these components helps the SVM algorithm classify data points into different classes while ensuring optimal separation with a maximized margin.

Linear SVM: In cases where data is linearly separable, a linear SVM can find a straight line (or hyperplane in higher dimensions) that best separate the classes.

Non-Linear SVM: For non-linearly separable data, SVM can employ a kernel trick, transforming the data into higher dimensions, making it possible to separate using a hyperplane.

Fig. Shown on Next Page for Non-linear SVM

Scanned with OKEN Scanner



Non-linear SVM.

Kernel types in SVM

① **Linear Kernel**: This is the simplest kernel, used when the data is linearly separable. It directly calculates the dot product of the input vectors.

$$K(x,y) = x \cdot y$$

where x and y are the input vectors.

② **Polynomial Kernel**: The polynomial kernel allows for non-linear decision boundaries by mapping the data into a higher-dimensional space using polynomial features.

$$K(x,y) = (x \cdot y + c)^d$$

where x and y are the input vectors, c is constant (often 0 or 1) and d is the degree of the polynomial.

③ **Radial Basis Function (RBF) Kernel/ Gaussian Kernel**: This is one of the most widely used kernels, effective for complex, non-linear classification problems. It maps the data into an infinite-dimensional space.

$$K(x,y) = \exp(-\gamma \|x-y\|^2)$$

where x and y are the input vectors, $\|x-y\|^2$ is the squared euclidean distance between x and y, and γ is a parameter controlling the influence of each training example.

Scanned with OKEN Scanner

④ Sigmoid Kernel: This kernel is derived from the hyperbolic tangent function and can be used for non-linear classification.

$$K(x, y) = \tanh(\alpha(x \cdot y) + c)$$

Where x and y are the input vector, α is a scaling parameter and c is an offset.

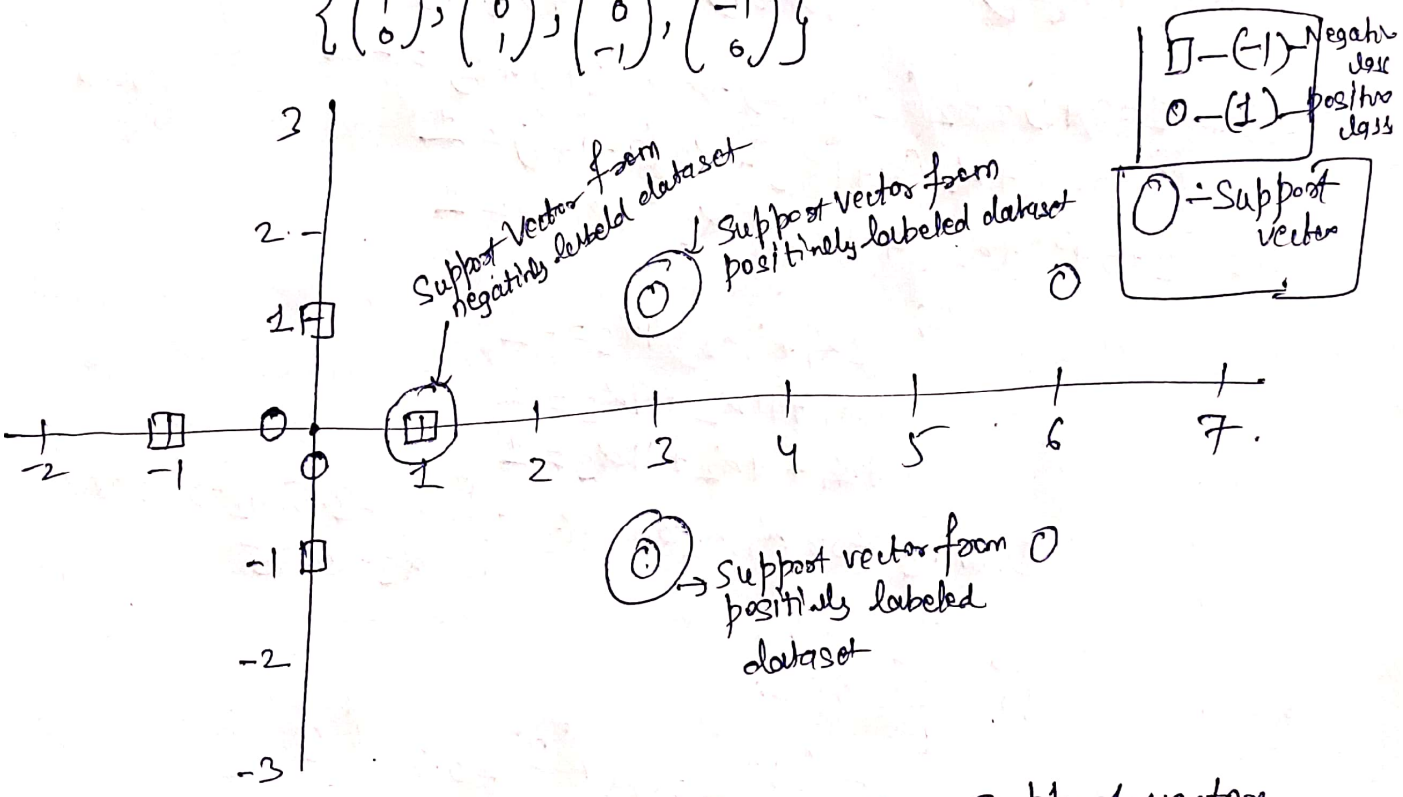
Support vector Machines - Linear Example

Suppose we have the following positively labeled data points

$$\left\{\begin{pmatrix}3\\1\end{pmatrix}, \begin{pmatrix}3\\-1\end{pmatrix}, \begin{pmatrix}6\\1\end{pmatrix}, \begin{pmatrix}6\\-1\end{pmatrix}\right\}$$

and the following negatively labeled data points

$$\left\{\begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}0\\1\end{pmatrix}, \begin{pmatrix}0\\-1\end{pmatrix}, \begin{pmatrix}-1\\0\end{pmatrix}\right\}$$



By inspection, it is obvious that there are three support vectors.

$$\left\{S_1 = \begin{pmatrix}1\\0\end{pmatrix}, S_2 = \begin{pmatrix}3\\1\end{pmatrix}, S_3 = \begin{pmatrix}3\\-1\end{pmatrix}\right\}$$

Each vector is augmented with a 1 as a bias input

So $S_1 = \begin{pmatrix}1\\0\end{pmatrix}$, then $\tilde{S_1} = \begin{pmatrix}1\\0\\1\end{pmatrix}$

Similarly

$S_2 = \begin{pmatrix}3\\1\end{pmatrix}$, then $\tilde{S_2} = \begin{pmatrix}3\\1\\1\end{pmatrix}$ and $S_3 = \begin{pmatrix}3\\-1\end{pmatrix}$ and $\tilde{S_2} = \begin{pmatrix}3\\-1\\1\end{pmatrix}$

Scanned with OKEN Scanner

We need to calculate three variables α_1, α_2 and α_3.

$$\alpha_1 \tilde{s_1} \cdot \tilde{s_1} + \alpha_2 \tilde{s_2} \cdot \tilde{s_1} + \alpha_3 \tilde{s_3} \cdot \tilde{s_1} = -1$$
$$\alpha_1 \tilde{s_1} \cdot \tilde{s_2} + \alpha_2 \tilde{s_2} \cdot \tilde{s_2} + \alpha_3 \tilde{s_3} \cdot \tilde{s_2} = 1$$
$$\alpha_1 \tilde{s_1} \cdot \tilde{s_3} + \alpha_2 \tilde{s_2} \cdot \tilde{s_3} + \alpha_3 \tilde{s_3} \cdot \tilde{s_3} = +1$$

$$\therefore \alpha_1 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}\begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix}\begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix}\begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} = -1$$

$$\Rightarrow \alpha_1 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}\begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix}\begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix}\begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} = 1$$

$$\alpha_1 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}\begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix}\begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix}\begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} = 1$$

$$\Rightarrow \alpha_1(1+0+1) + \alpha_2(3+0+1) + \alpha_3(3+0+1) = -1$$
$$\alpha_1(3+0+1) + \alpha_2(9+1+1) + \alpha_3(9-1+1) = 1$$
$$\alpha_1(3+0+1) + \alpha_2(9-1+1) + \alpha_3(9+1+1) = 1$$

$$\therefore 2\alpha_1 + 4\alpha_2 + 4\alpha_3 = -1$$
$$4\alpha_1 + 11\alpha_2 + 9\alpha_3 = 1$$
$$4\alpha_1 + 9\alpha_2 + 11\alpha_3 = 1$$

Solving for α_1, α_2 and α_3 $\therefore$ $\boxed{\alpha_1 = -3.5,\ \alpha_2 = 0.25 \text{ and } \alpha_3 = 0.75}$

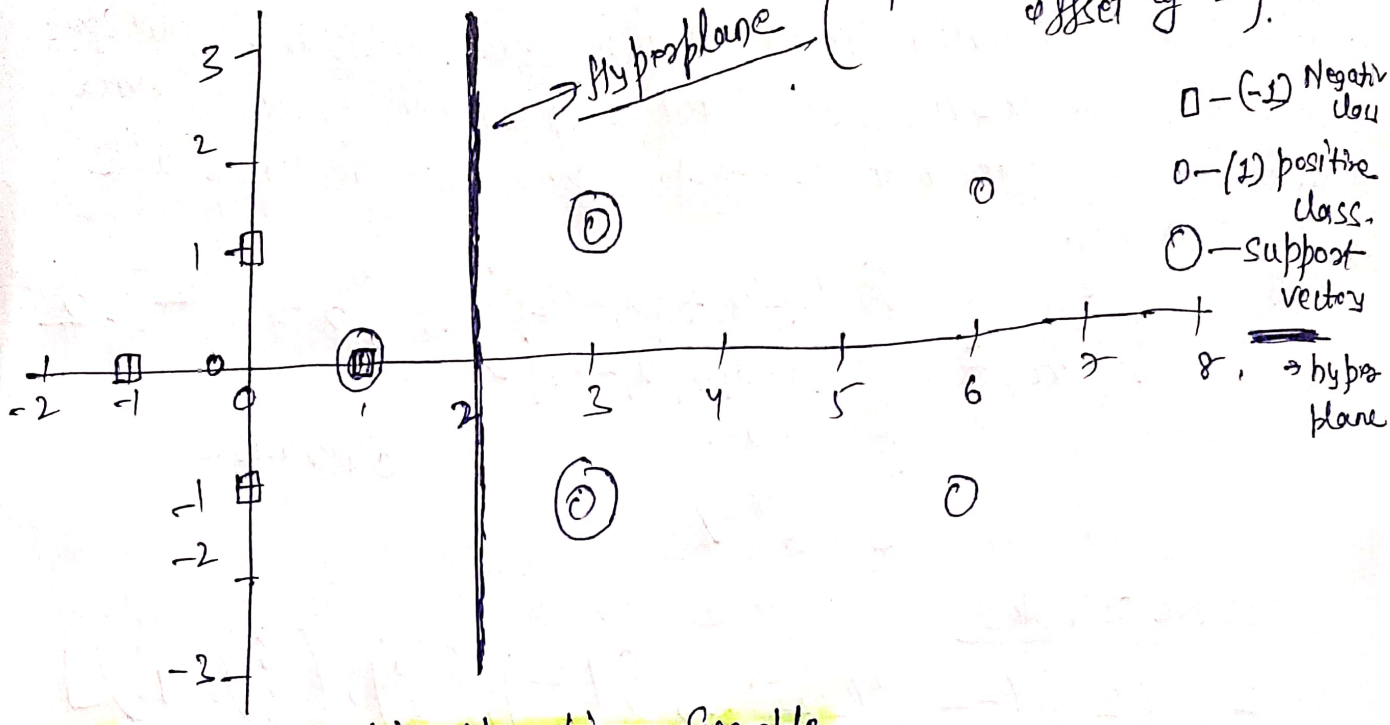
Now
$$\tilde{w} = \sum_i \alpha_i \tilde{s_i}$$
$$= -3.5\begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + 0.75\begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} + 0.75\begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix}$$
$$\Rightarrow \begin{pmatrix} 1 \\ 0 \\ -2 \end{pmatrix}$$

Finally, remembering that our vectors are augmented with a bias. We can equate the last entry in $\tilde{w}$ as the hyperplane offset b and write the separating hyperplane equation $y = wx + b$

with $w = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $b = -2$.

Scanned with OKEN Scanner

Therefore the final graph will look like.



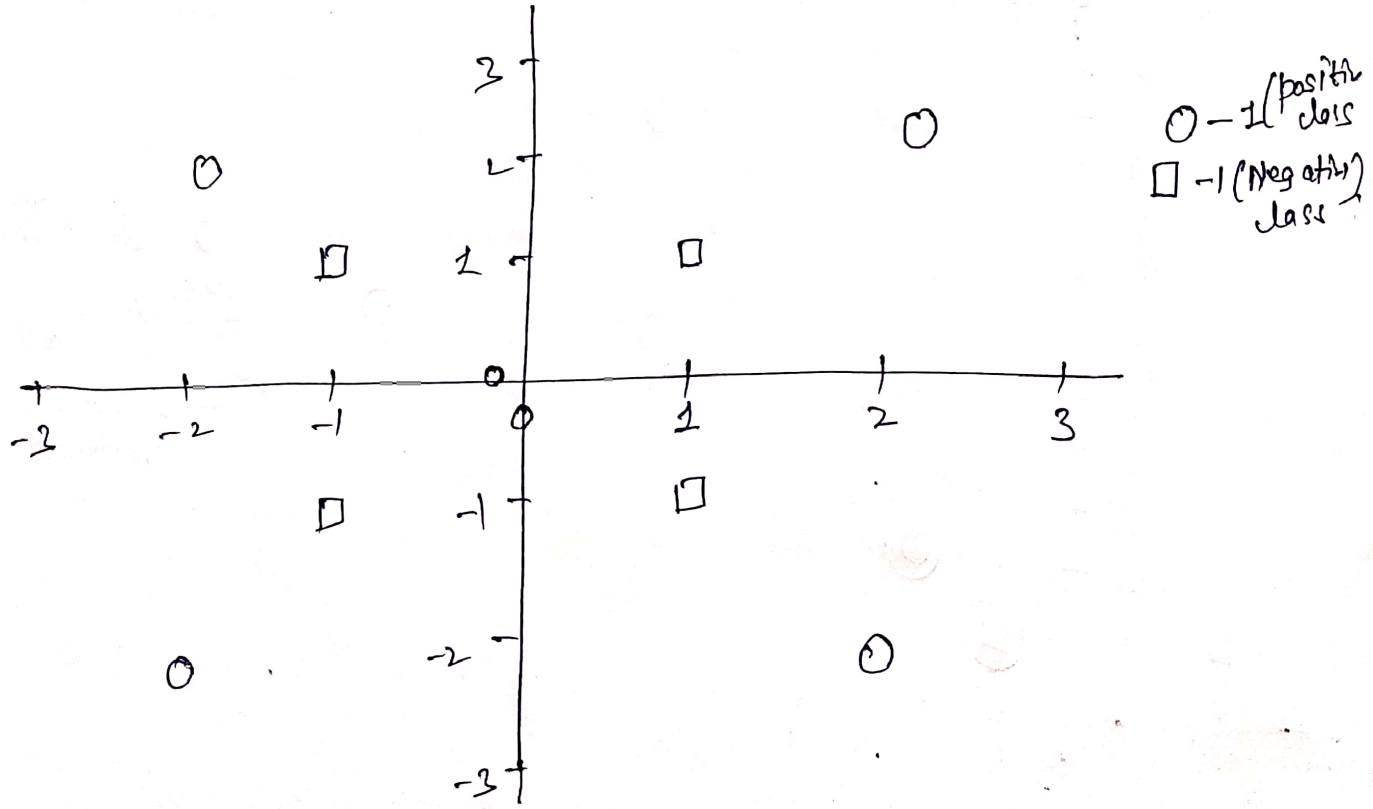
Support Vector Machine – Non-linear Example

Suppose we are given the following positively labeled data points.

$$\left\{\begin{pmatrix}2\\2\end{pmatrix}, \begin{pmatrix}2\\-2\end{pmatrix}, \begin{pmatrix}-2\\-2\end{pmatrix}, \begin{pmatrix}-2\\2\end{pmatrix}\right\}$$

and the following negatively labeled data points

$$\left\{\begin{pmatrix}1\\1\end{pmatrix}, \begin{pmatrix}-1\\1\end{pmatrix}, \begin{pmatrix}-1\\-1\end{pmatrix}, \begin{pmatrix}1\\-1\end{pmatrix}\right\}$$



Scanned with OKEN Scanner

(1) The goal is to discover a separating hyperplane that accurately discriminates the two classes.

(2) of course, it is obvious that no such hyperplane exists in the input space

(3) Therefore non-linear SVM is used (that is, it is needed to convert data from one feature space to another feature space, where the classes are linearly separable).

$$\therefore \phi\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{cases} \begin{pmatrix} 4 - x_2 + |x_1 - x_2| \\ 4 - x_1 + |x_1 - x_2| \end{pmatrix} & \text{if } \sqrt{x_1^2 + x_2^2} > 2 \\ \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} & \text{otherwise} \end{cases}$$

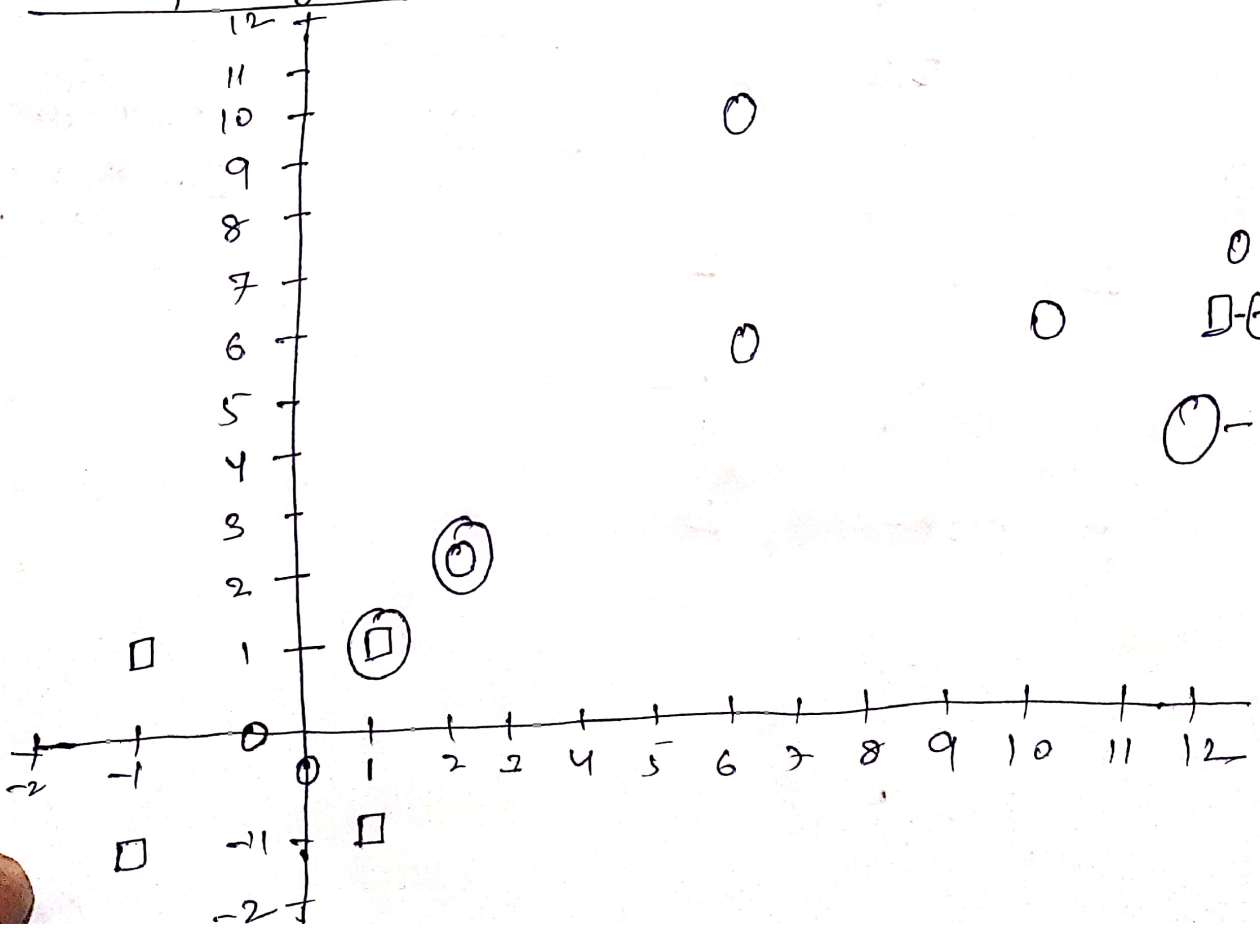
∴ Positive samples

$$\left\{ \begin{pmatrix} 2 \\ 2 \end{pmatrix}, \begin{pmatrix} 2 \\ -2 \end{pmatrix}, \begin{pmatrix} -2 \\ -2 \end{pmatrix}, \begin{pmatrix} -2 \\ 2 \end{pmatrix} \right\} \rightarrow \left\{ \begin{pmatrix} 2 \\ 2 \end{pmatrix}, \begin{pmatrix} 10 \\ 6 \end{pmatrix}, \begin{pmatrix} 6 \\ 6 \end{pmatrix}, \begin{pmatrix} 6 \\ 10 \end{pmatrix} \right\}$$

Negative Examples

$$\left\{ \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \begin{pmatrix} -1 \\ -1 \end{pmatrix}, \begin{pmatrix} -1 \\ 1 \end{pmatrix} \right\} \rightarrow \left\{ \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \begin{pmatrix} -1 \\ -1 \end{pmatrix}, \begin{pmatrix} -1 \\ 1 \end{pmatrix} \right\}$$

Now plotting these points



Scanned with OKEN Scanner

Now identifying the support vectors

$\left\{ S_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, S_2 = \begin{pmatrix} 2 \\ 2 \end{pmatrix} \right\}$. Each vector is augmented with a 1 bias input

Therefore $\tilde{S_1} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$ and $\tilde{S_2} = \begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix}$

$\therefore \alpha_1 \tilde{S_1} \cdot \tilde{S_1} + \alpha_2 \tilde{S_2} \cdot \tilde{S_1} = -1 \Rightarrow \alpha_1 \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = -1$

$\alpha_1 \tilde{S_1} \cdot \tilde{S_2} + \alpha_2 \tilde{S_2} \cdot \tilde{S_2} = -1 \quad \alpha_1 \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix} \begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix} = 1$

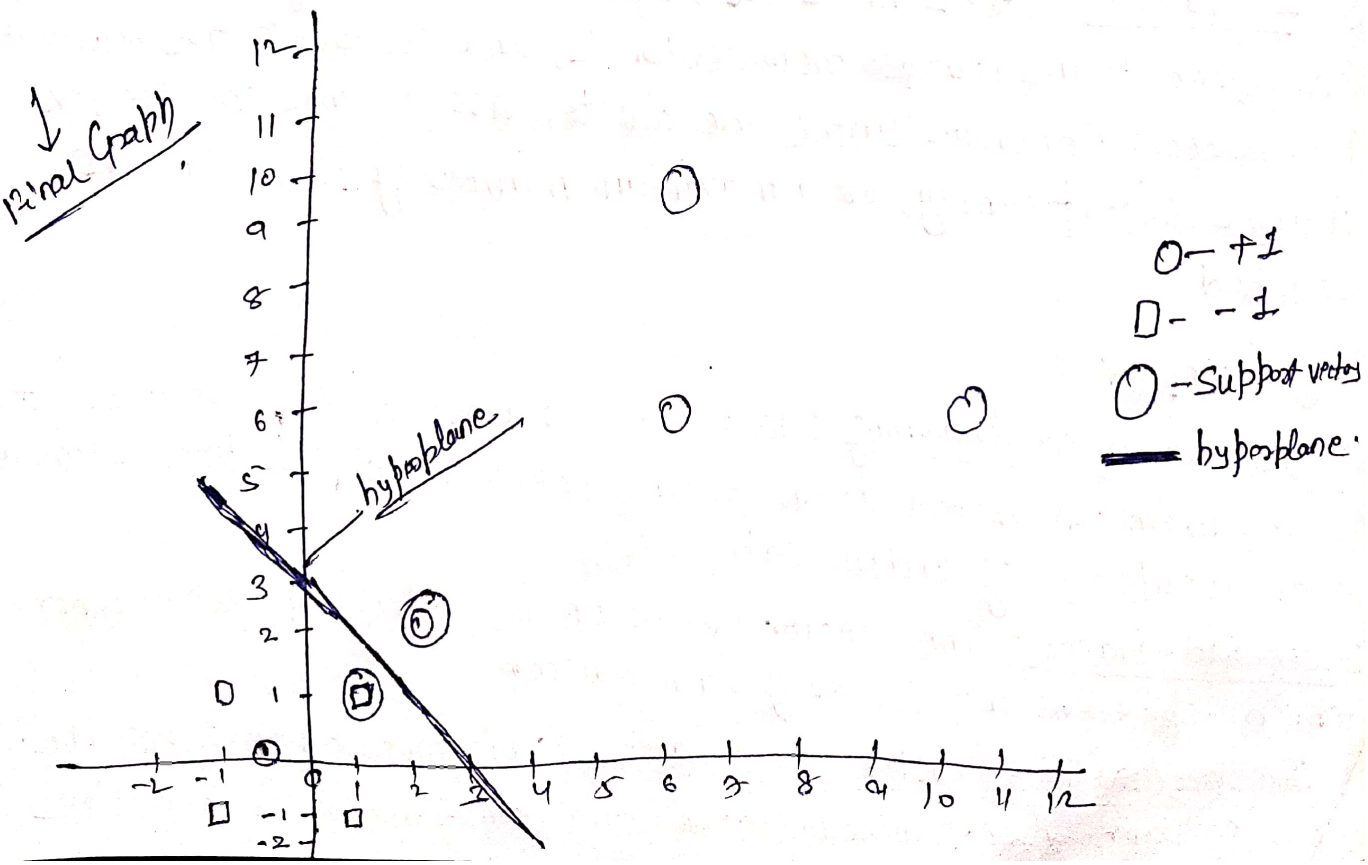
$\Rightarrow \left\{ \begin{array}{l} \alpha_1(1+1+1) + \alpha_2(2+2+1) = -1 \\ \therefore \alpha_1(2+2+1) + \alpha_2(4+4+1) = 1 \end{array} \right\} \Rightarrow$ Solving for α_1 and α_2

$\therefore \left\{ \begin{array}{l} 3\alpha_1 + 5\alpha_2 = -1 \\ 5\alpha_1 + 9\alpha_2 = 1 \end{array} \right\} \therefore \boxed{\alpha_1 = -7 \text{ and } \alpha_2 = 4}$

$\therefore \tilde{w} = \sum_i \alpha_i \tilde{S_i} = -7 \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} + 4 \begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ -3 \end{pmatrix}$

Remembering that our vectors are augmented with a bias. We can equate the last entry in $\tilde{w}$ as the hyperplane offset b and write the separating hyperplane equation $y = wx + b$ ∴ with $w = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ and $b = -3$

↓ Final Graph.



Scanned with OKEN Scanner

Unsupervised learning:

K-Means clustering: K-means clustering is an unsupervised machine learning algorithm used to partition a dataset into a pre-defined number of clusters, denoted by (K). This algorithm aims to group similar data points together, minimizing the sum of squared distances between each data point and the centroid of its assigned cluster.

How K-means clustering works:

① **Initialization:** The algorithm begins by randomly selecting (K) data points from the dataset to serve as initial cluster centroids.

② **Assignment Step:** Each data point in the dataset is assigned to the nearest centroid, forming initial clusters. The distance is typically calculated using Euclidean distance.

③ **Update step:** After reassigning all data points, the centroids of each cluster are recalculated. The new centroid for a cluster becomes the mean position of all data points assigned to that cluster.

④ **Iteration:** Step 2 and 3 are repeated iteratively. Data points are reassigned to the nearest new centroids, and centroids are recalculated. This process continues until the cluster assignments no longer change significantly, or a maximum number of iterations is reached.

Key Concepts:

① **Unsupervised learning:** K-means is unsupervised because it works with unlabeled data, finding patterns and structures without prior knowledge of output categories.

② **Centroid-Based:** The cluster relies on the concept of cluster centroids which represent the center of each cluster.

③ **Inertia (within-cluster Sum of Square):** This metric measures the sum of squared distances between data points and their respective

cluster centroids. The goal of K-means is to minimize inertia, indicating denser, more cohesive clusters.

④ Choosing K: Determining the optimal number of clusters (K) is crucial. Methods like the "elbow method", which plots inertia against different 'K' values to identify a point of diminishing returns, are commonly used.

Applications:

① Customer Segmentation: Grouping customers based on purchasing behavior or demographics.

② Image Segmentation: Dividing images into distinct regions based on pixel characteristics.

③ Document classification: Categorizing documents into themes or topics.

④ Anomaly Detection: Identifying unusal data points that do not fit into any cluster.

Example of K-Means clustering algorithm

(Q) Apply K(=2)-Means algorithm over the data (185, 72), (170, 56), (168, 60), (179, 68), (182, 72), (188, 77) up to two iterations and show the clusters. Initialize the first two objects as the initial centroids.

Sol: Given, the no. of clusters to be created (K) = 2 say C_1 and C_2,
no. of iterations = 2
and the data points can be represented in tabular form.

Instance	X	Y
1	185	72
2	170	56
3	168	60
4	179	68
5	182	72
6	188	77

Scanned with OKEN Scanner

also, first two objects as initial centroids:

∴ Centroid for first cluster $C1 = (185, 72)$
Centroid for second cluster $C2 = (170, 56)$

Iteration 1: Calculating the similarity between these centroid. points and the other points using Euclidean distance.

$$\therefore d(X, Y) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$\therefore d(C1, 3) \Rightarrow \sqrt{(185-168)^2 + (72-60)^2} = \sqrt{289+144} = \sqrt{433} = 20.80$

$d(C2, 3) \Rightarrow \sqrt{(170-168)^2 + (56-60)^2} = \sqrt{4+16} = \sqrt{20} = 4.472$

So, data point 3 belongs to C2

$\therefore d(1, 4) = \sqrt{(185-179)^2 + (72-68)^2} = \sqrt{36+16} = \sqrt{52} = 7.21$

$d(C2, 4) = \sqrt{(170-179)^2 + (56-68)^2} = \sqrt{81+144} = \sqrt{225} = 15$

So, data point 4 belongs to C1.

$\therefore d(C1, 5) = \sqrt{(185-188)^2 + (72-72)^2} = \sqrt{9} = 3$

$\therefore d(C2, 5) = \sqrt{(170-188)^2 + (56-72)^2} = \sqrt{324+256} = \sqrt{400} = 20$

So, data point 5 belongs to C1.

$\therefore d(C1, 6) = \sqrt{(185-188)^2 + (72-77)^2} = \sqrt{9+25} = \sqrt{34} = 5.83$

$d(C2, 6) = \sqrt{(170-188)^2 + (56-77)^2} = \sqrt{324+441} = \sqrt{765} = 27.65$

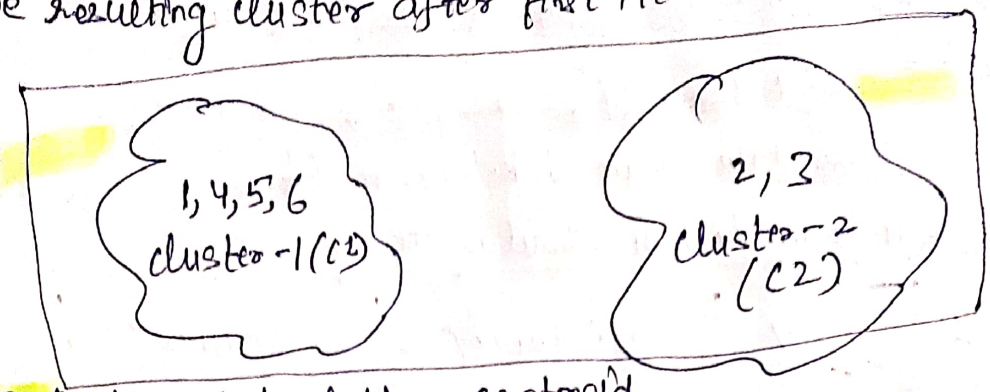
So, data point 6 belongs to C1

Representing the above information in tabular form

Instance	X	Y	Distance (C1)	Distance (C2)	Cluster
1	185	72	0	0	C1
2	170	56	0	0	C2
3	168	60	$\sqrt{433}$ (20.80)	$\sqrt{20}$ (4.472)	C2
4	179	68	$\sqrt{52}$ (7.21)	$\sqrt{225}$ (15)	C1
5	182	72	$\sqrt{9}$ (3)	$\sqrt{400}$ (20)	C1
6	188	77	$\sqrt{34}$ (5.83)	$\sqrt{765}$ (27.65)	C1

Scanned with OKEN Scanner

The resulting cluster after first iteration is



Data points cluster

Iteration 2: Now calculating centroid for each cluster

Centroid for first cluster $C1 = \left(\frac{185+179+182+188}{4}, \frac{72+68+72+79}{4}\right)$

$= (183.5, 72.25)$.

Centroid for second cluster $C2 = \left(\frac{170+168}{2}, \frac{56+60}{2}\right) = (169, 58)$

Now, again calculating similarity between these centroid points and all the other data points.

$\therefore d(C1, 1) = \sqrt{(183.5-185)^2+(72.25-72)^2} = 1.5207$

$d(C2, 1) = \sqrt{(169-185)^2+(58-72)^2} = 21.2603$

So, data point 1 belongs to C1.

$d(C1, 2) = \sqrt{(183.5-170)^2+(72.25-56)^2} = 21.1261$

$d(C2, 2) = \sqrt{(169-170)^2+(58-56)^2} = 2.2361$

So, data point 2 belongs to C2

$d(C1, 3) = \sqrt{(183.5-168)^2+(72.25-60)^2} = 19.7562$

$d(C2, 3) = \sqrt{(169-168)^2+(58-60)^2} = 2.2361$

$\therefore$ So, data point 3 belongs to C2.

$d(C1, 4) = \sqrt{(183.5-179)^2+(72.25-68)^2} = 6.1893$

$d(C2, 4) = \sqrt{(169-179)^2+(58-68)^2} = 14.1421$

$\therefore$ So, data point 4 belongs to C1

$d(C1, 5) = \sqrt{(183.5-182)^2+(72.25-72)^2} = 1.5207$

$d(C2, 5) = \sqrt{(169-182)^2+(58-72)^2} = 19.1050$

$\therefore$ So, data point 5 belongs to C1

Scanned with OKEN Scanner

$d(C1,6) = \sqrt{(183.5-188)^2+(72.25-77)^2} = 6.5431$

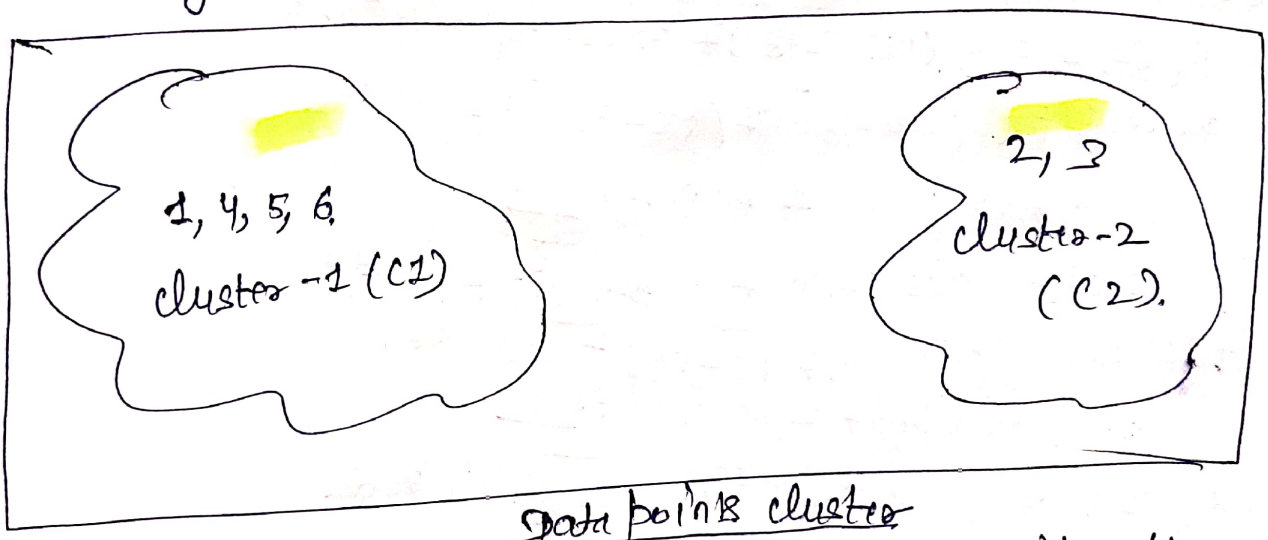
$d(C2,6) = \sqrt{(169-188)^2+(58-77)^2} = 26.8701$

∴ So data point 6 belongs to C1.

Representing above information in tabular form.

Instance	X	Y	Distance(C1)	Distance(C2)	cluster
1	185	72	1.5207	21.2603	C1
2	170	56	21.1261	2.2361	C2
3	168	60	19.7563	2.2361	C2
4	179	68	6.1897	14.1421	C1
5	182	72	1.5207	19.105	C1
6	188	77	6.5431	26.8701	C1

The resulting cluster after second iteration is:



Data points cluster

Since, the clustering doesn't change after second iteration, so terminate the iteration even if the question doesn't say so.

Naive Bayes classifier Cont: (example)

Today (SUNNY, RAIN)

⇒ P(Yes | Today = SUNNY, RAIN)

⇒ P[Sunny | Yes) X P(Rain | Yes) X P(Yes.

⇒ $\frac{2}{3} \times \frac{3}{9} \times \frac{9}{14} = \frac{1}{21} = .0476$

$P(No|Today = SUNNY, RAIN)$

$\Rightarrow P(Sunny|No) \times P(RAIN|No) \times P(No)$

$\Rightarrow \frac{3}{5} \times \frac{2}{5} \times \frac{5}{14} = \frac{3}{35} = .0857$

$\Rightarrow$ Normalizing the probabilities

$P(Yes|Today) = \frac{.0476}{.0476 + .0857} = \frac{.0476}{.1333} = .357$

$P(No|Today) = \frac{.0857}{.0476 + .0857} = .642$

Today (SUNNY, RAIN) = No

Today (Overcast, COOL)

$\Rightarrow P(Yes|Today = OVERCAST, COOL)$

$\Rightarrow P(Overcast|Yes) \times P(Cool|Yes) \times P(Yes)$

$\Rightarrow \frac{4}{4} \times \frac{3}{9} \times \frac{9}{14} = \frac{2}{21} \Rightarrow .0952$

$P(No|Today = OVERCAST, COOL)$

$\Rightarrow P(overcast|No) \times P(Cool|No) \times P(No)$

$\Rightarrow \frac{0}{5} \times \frac{1}{5} \times \frac{5}{14} = 0.$

$\therefore$ Normalizing

$P(Yes|Today) = \frac{.0952}{.0952 + 0} = 1$

$P(No|Today) = \frac{0}{.0952 + 0} = 0.$

$\therefore$ **Today (Overcast, Cool) = Yes**

Scanned with OKEN Scanner

Hierarchical Clustering

① **Agglomerative clustering**: Agglomerative is a bottom up, data hierarchial machine learning approach that starts with each data point in its own cluster and iteratively merges the closest or clusters until all points are in one cluster. It is an unsupervised learning method that groups data based on similarity, producing a tree like structure called a dendrogram that shows the relationships between clusters at different levels. Key steps involve calculating a distance matrix, merging the closest clusters based on a linkage criterion, and updating the matrix until a stopping condition is met.

How it works

① **Initialization**: Each data point is initially treated as its own separate cluster.

② **Distance calculation**: A distance matrix is computed to find the distance between every pair of data points. Common distance metrics include min, euclidean or Manhattan.

③ **Merging**: The two clusters with the smallest distance between them are merged into a single new cluster.

④ **Matrix update**: The distance matrix is updated to reflect the new cluster. This involves calculating the distance between the new cluster and all other existing clusters, using a chosen linkage criterion. (e.g, single, complete or average linkage).

⑤ **Iteration**: The process of merging and updating is repeated until all data points are in a single cluster or another stopping criterion is met, such as achieving a desired number of clusters

Dendrogram

Scanned with OKEN Scanner

① The merging process creates a dendrogram, which is a tree like diagram that visualizes the hierarchical relationships between clusters.

② The height of each image in the dendrogram indicates the distance or dissimilarity between the two clusters being merged.

③ By "cutting" the dendrogram at certain height, you can determine the number of clusters for the dataset.

Example of Agglomerative Hierarchical clustering

(a) Consider the following set of 6 one dimensional data points:

18, 22, 25, 27, 42, 43

(a) Apply the agglomerative hierarchical clustering algorithm to build the hierarchical clustering algorithm.

(b) Merge the clusters using Min distance to update the proximity matrix accordingly.

(c) clearly show the proximity matrix corresponding to each iteration of the algorithm.

Sol:

Step1 : Putting data points in horizontal and vertical axis

	18	22	25	27	42	43
18	0	4	7	9	24	25
22	4	0	3	5	20	21
25	7	3	0	2	17	18
27	9	5	2	0	15	16
42	24	20	17	15	0	1
43	25	21	18	16	1	0

data points

data point →

distances

By looking the distance matrix, the distance between 42 and 43 is minimum (i.e 1). Therefore merging 42 and 43

Scanned with OKEN Scanner

At the first step, the cluster is created by merging 42 and 43 So the 43 rows and columns are removed and the matrix will be look like (in step 2).

Step 2

	18	22	25	27	42,43
18	0	4	7	9	24
22	4	0	3	5	20
25	7	3	0	2	17
27	9	5	2	0	15
42,43	24	20	17	15	0

Now the distance between 25 and 27 is minimum (i'e 2). Therefore merging 25 and 27. This will create another cluster by merging 25 and 27, so the 27 rows and columns are removed and the matrix will look like (in step 3)

Step 3

	18	22	25,27	42,43
18	0	4	7	24
22	4	0	3	20
25,27	7	3	0	15
42,43	24	20	15	0

Now the distance between 22 and (25, 27) is minimum (3). Therefore merging 22 and (25, 27). This will create another cluster by merging 22 and (25, 27), so the (25, 27) rows and columns are removed. and the matrix will look like (in step 4)

step 4

	18	22,25,27	42,43
18	0	4	24
22,25,27	4	0	15
42,43	24	15	0

Scanned with OKEN Scanner

Step-4 : Now the distance between 18 and (22, 25, 27) is minimum (i.e 4) therefore merging 18 and (22, 25, 27). This will create another cluster by merging 18 and (22, 25, 27). So the (22, 25, 27) rows and columns are removed and the matrix will look like in step 5.

Step 5

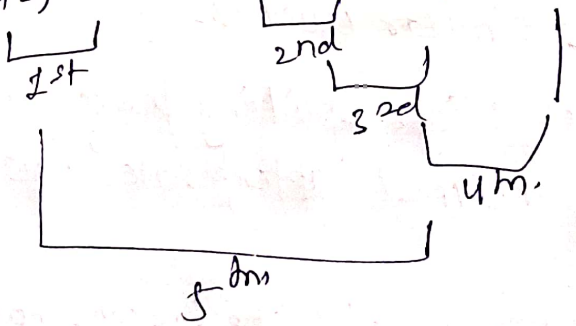
	18, 22, 25, 27	42, 43
18, 22, 25, 27	0	15
42, 43	15	0

Finally combining 18, 22, 25, 27 and 42, 43 to get final matrix. The matrix will be look like in (step 6)

Step 6

	18, 22, 25, 27, 42, 43
18, 22, 25, 27, 42, 43	0.

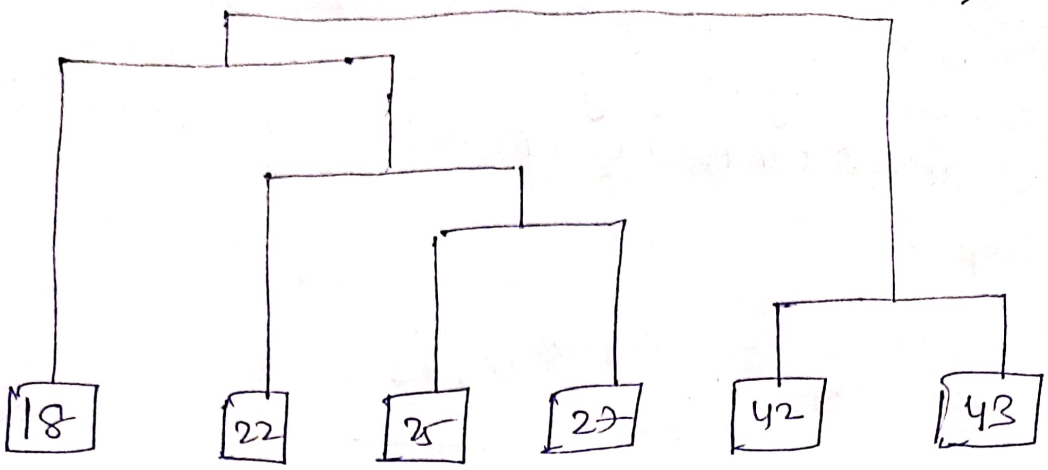
The joining of the data elements in cluster wise is as follows ((42, 43), ((25, 27), 22), 18))



The dendrogram is given in the next page

Scanned with OKEN Scanner

Dendrogram for ((42, 43), ((25, 27), 22), 18))



The Dendrogram of the above sequence (for Agglomerative clustering)

(b) Divisive clustering: Divisive clustering is a top-down approach to clustering that starts with all datapoints in a single cluster and recursively split into smaller, more homogeneous clusters until a stopping criterion is met. It is opposite of agglomerative clustering.

How it works

(1) Starts with one cluster: All data points are initially placed in a single cluster.

(2) Split the cluster: The algorithm identifies the cluster with the largest "diameter" or the most dissimilar points and splits it into two new, less similar clusters.

(3) Repeat the process: The splitting process is repeated for each new cluster, choosing the most "appropriate" cluster to divide at each step.

(4) Stop when a Condition is met: The process continues until a stopping condition is reached, such as each data point being in its own cluster or a predefined number of clusters being created.

Scanned with OKEN Scanner

Therefore the sequence is (18, 22, 25, 27, 42, 43)

Example: For the given dataset {A, B, C, D, E} and the distance matrix in a table apply divisive clustering algorithm. to form clusters.

Table Given

	A	B	C	D	E
A	0	1	2	2	3
B	1	0	2	4	3
C	2	2	0	1	5
D	2	4	1	0	3
E	3	3	5	3	0

Scanned with OKEN Scanner

Step 1: Initially $C_1 = \{A, B, C, D, E\}$

Step 2: $C_i = C_1$ and $C_j = \phi$

Step 3: Initial Iteration

- Let us calculate the average dissimilarities of the objects in C_i with the other objects in C_i.

- Average dissimilarity of A

$$\Rightarrow \frac{1}{4}(d(a,b)+d(a,c)+d(a,d)+d(a,e)) = \frac{1}{4}(1+2+2+3) = \frac{8}{4} = 2.00$$

- Average dissimilarity of B.

$$= \frac{1}{4}(d(b,a)+d(b,c)+d(b,d)+d(b,e)) = \frac{1}{4}(1+2+4+3) = \frac{10}{4} = 2.50$$

- Average dissimilarity of C

$$= \frac{1}{4}(d(c,a)+d(c,b)+d(c,d)+d(c,e)) = \frac{1}{4}(2+2+1+5) = \frac{10}{4} = 2.50$$

- Average dissimilarity of D

$$= \frac{1}{4}(d(d,a)+d(d,b)+d(d,c)+d(d,e)) = \frac{1}{4}(2+4+1+3) = \frac{10}{4} = 2.50$$

- Average dissimilarity of E

$$= \frac{1}{4}(d(e,a)+d(e,b)+d(e,c)+d(e,d)) = \frac{1}{4}(3+3+5+3) = \frac{14}{4} = 3.50$$

$\therefore$ **Average dissimilarity**

2	2.5	2.5	2.5	3.5
A	B	C	D	E

$\therefore$ E is greater than all the other, So E is moved to new cluster C_j

$\therefore$

$C_i = \{A, B, C, D\}$ and $C_j = \{E\}$

Step 4 Is there any possibility to move data points from $\{A, B, C, D\}$ to $\{E\}$

to do that Calculating the distance for each object $x \in C_i$:

$\therefore D_x$ = avg dissimilarity of x with objects in C_i - dissimilarity of x with C_j

$\therefore$ Second iteration

$D_a = \frac{1}{3}(d(a,b) + d(a,c) + d(a,d) - d(a,e))$

$= \frac{1}{3}(1+2+1) - 3 = \frac{5}{3} - 3 = -\frac{4}{3} \approx -1.33$

$D_b = \frac{1}{3}(d(b,a) + d(b,c) + d(b,d) - d(b,e))$

$= \frac{1}{3}(1+2+4) - 3 = \frac{7}{3} - 3 = -\frac{2}{3} \approx -0.67$

$D_c = \frac{1}{3}(d(c,a) + d(c,b) + d(c,d) - d(c,e)) = \frac{1}{3}(2+2+1) - 5 = \frac{5}{3} - 5 = -\frac{10}{3}$

≈ -3.33

$D_d = \frac{1}{3}(d(d,a) + d(d,b) + d(d,c) - d(d,e))$

$= \frac{1}{3}(2+4+1) - 3 = \frac{7}{3} - 3 = -\frac{2}{3} \approx -0.67$

$\therefore$ $D_a = -1.33$, $D_b = -0.67$, $D_c = -3.33$, $D_d = -0.67$

As all are negative therefore no object is moved from C_i to E $\therefore$ $C_i = \{A, B, C, D\}$ and $C_j = \{E\}$

Step 5

Now considering $\{A, B, C, D\}$

$\therefore C_i = \{A, B, C, D\}$

$\cdot C_i = C_1$ and $\cdot$ and $C_j = \phi$

Initial Iteration

- Average dissimilarity of A

$= \frac{1}{3}(d(a,b) + d(a,c) + d(a,d)) = \frac{1}{3}(1+2+2) = \frac{5}{3} \approx 1.67$

Scanned with OKEN Scanner

• Average dissimilarity of B

$= \frac{1}{3}(d(b,a)+d(b,c)+d(b,d)) = \frac{1}{3}(1+2+4) = \frac{7}{3} \approx 2.33$

• Average dissimilarity of C

$= \frac{1}{3}(d(c,a)+d(c,b)+d(c,d)) = \frac{1}{3}(2+2+1) = \frac{5}{3} \approx 1.67$

• Average dissimilarity of D

$= \frac{1}{3}(d(d,a)+d(d,b)+d(d,c)) = \frac{1}{3}(2+4+1) = \frac{7}{3} \approx 2.33$

$D_a = 1.67$
$D_b = 2.33$
$D_c = 1.67$
$D_d = 2.33$

• The highest average dissimilarity is 2.33 and both B and D have this value. There abbitarily B is chosen and put in into the another cluster C_j

Step 5

Remaing Iterations

$C_i = \{A, C, D\}$ and $C_j = \{B\}$

Calculate the distance for each object $x \in C_i$:

D_x = avg dissimilarity of x with objects in C_i - dissimilarity of x with C_j

$\therefore D_a = \frac{1}{2}(d(a,c)+d(a,d)) - \frac{1}{1}(d(a,b))$

$= \frac{1}{2}(2+2) - 1 = \frac{4}{2} - 1 = 2 - 1 = 1.0$

$D_c = \frac{1}{2}(d(c,a)+d(c,d)) - \frac{1}{1}(d(c,b))$

$= \frac{1}{2}(2+1) - 2 = \frac{3}{2} - 2 = 1.5 - 2 = -0.5$

$D_d = \frac{1}{2}(d(d,a)+d(d,c)) - \frac{1}{1}(d(d,b))$

$= \frac{1}{2}(2+1) - 4 = \frac{3}{2} - 4 = 1.5 - 4 = -2.5$

Scanned with OKEN Scanner

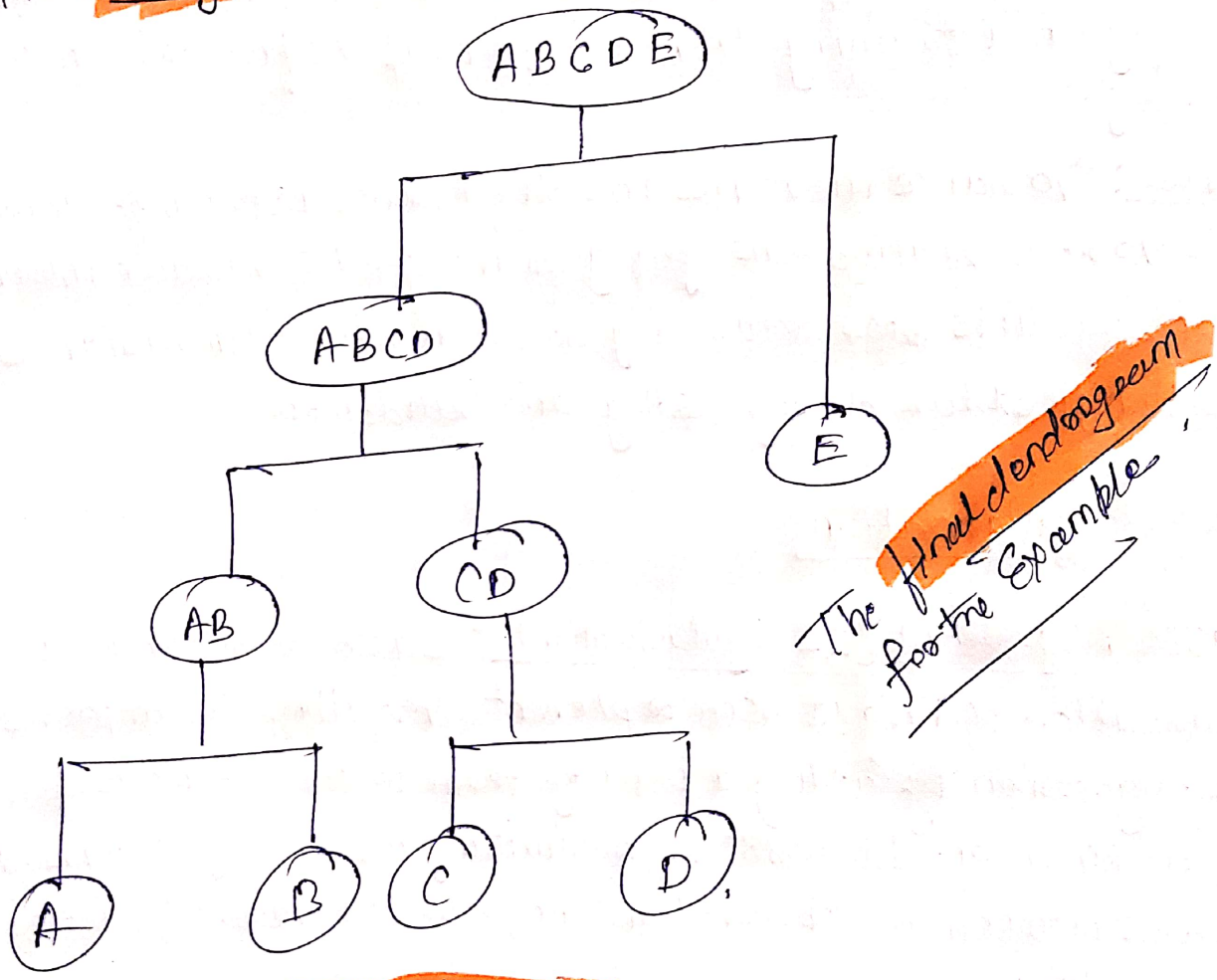
$\therefore$ $D_a = 1$, $D_c = -0.5$ and $D_d = -2.5$

Highest average dissimilarity is 1, for object A. So we move A to the new cluster C_j $\therefore C_i = \{C, D\}$ and $C_j = \{B, A\}$

$\therefore$ **Final clusters** $\therefore \{B, A\}, \{C, D\}$ and $\{E\}$

$= \{A, B\}, \{C, D\}$ and $\{E\}$

The **dendrogram** will look like this



The final dendrogram for the example

Regularization and Bias/Variance: Regularization is a set of methods for reducing overfitting in machine learning models. Typically regularization trades a marginal decrease in training accuracy for an increase in generalizability.

Regularization encompasses a range of techniques to correct for overfitting in machine learning models. As such regularization is a

Scanned with OKEN Scanner

method for increasing a model's generalizability – that is ability to produce accurate predictions on new datasets. Regularization provides this increased generalizability at the sake of increased training error. In other words, regularization methods typically lead to less accurate predictions on training data but more accurate predictions on test data.

Bias–Variance tradeoff :

Bias : Bias measures the average difference between predicted values and true values. As bias increases, a model predicts less accurately on a training dataset. High bias refers to high error in training.

Variance : Variance measures the difference between predictions across various realizations of a given model. As variance increases, a model predicts less accurately on unseen data. High variance refers to high error during testing and validation.

Types of regularization

① **Lasso Regression (L1 regularization)** : Lasso regression is a regularization technique used to prevent overfitting. It improves linear regression by adding a penalty term to the standard regression equation. It works by minimizing the sum of squared differences between the observed and the predicted values by fitting a line to the data.

However in real-world datasets features have strong correlations with each other known as multicollinearity where Lasso Regression actually helps. For eg if we are predicting house prices based on features like location, square footage and number of bedrooms. Lasso Regression can identify most important features. It might determine that location and square footage are the key factors influencing

Scanned with OKEN Scanner

pare while others have less impact. By making the coefficient for the bedroom feature to zero, it simplies the model and improves its accuracy.

When implementing Lasso Regression penalty reduces variance by making the coefficients of less important features to zero. This prevents overfitting by ensuring model doesn't fit to noise in the data. However increasing regularization strength i.e. raising the lamda value can increase bias. This happens because a stronger penalty can cause the model to oversimplify making it unable to capture the true relation-ships in the data leading to underfitting. Thus the goal is to choose the right lambda value that balances both bias and variance through cross-validation.

(2) **Ridge Regression (L2 Regularization)** : is a technique used in linear regression to address the problem of multicollinearity among predictor variables. Multicollinearity occurs when independent variables in a regression model are highly correlated, which can lead to unreliable and unstable estimates of regression coefficients.

Overfitting occurs when a model becomes too complex and fits the noise in the training data, leading to a poor generalization on new data. Ridge regression combats overfitting by adding a penalty term (L2 regularization) to the ordinaly least square (OLS) objective function. This penalty discourages the model from using large values for the coefficients (the number multiplying the feature). It forces the model to keep these coefficients small. By making the coefficients smaller and closer to zero, Ridge regression simplifies the model and reduces its sensitivity to random fluctuations or noise in the data. This makes the model less likely to overfit and helps it perform better on new, unseen data, improving it overall accuracy and reliability.

Scanned with OKEN Scanner

For example - we are predicting house prices based on multiple features such as: square footage, number of bedrooms and age of the house:

$$Price = 1000 \cdot Size - 500 \cdot Age + Noise$$

- Ridge might adjust to it

$$Price = 800 \cdot Size - 300 \cdot Age + Less\ Noise$$

As lambda increases the model places more emphasis on shrinking the coefficients of highly correlated features, making their impact smaller and more stable. This reduces the effect of multicollinearity by preventing large fluctuations in coefficient estimates due to correlated predictors

(3) **Elastic Net Regression:** is a type of linear regression that adds two types of penalties, L1 (from Lasso) and L2 (from Ridge) to its cost function. This helps picking out important features by setting some coefficients to zero and also handle situations where some features are highly similar or correlated. This makes the model less likely to overfit and more stable, especially when working with lots of features or data where some variables are related to each other

Ensemble Methods:

① **Bagging:** Bagging is a popular ensemble learning technique that focus on reducing variance and improving the stability of machine learning models. The term "bagging" is derived from the idea of creating multiple subsets or bags of the training data through a process known as bootstrapping. Bootstrapping involves randomly sampling the dataset with replacement to generate multiple subsets of same size as the original data. Each of these subsets then used to

Scanned with OKEN Scanner

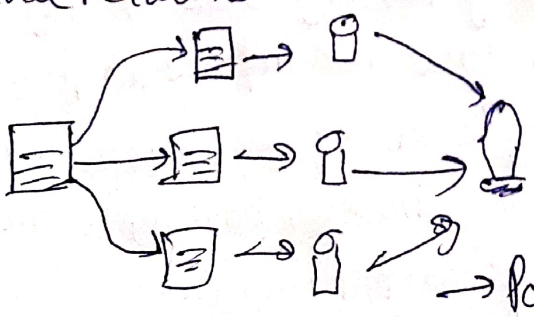
train a base learner independently.

One of the primary goals of bagging is to reduce overfitting by exposing each base learner to slightly different variations of the training data. Since each subset is created by sampling with replacement, some instances may be duplicated while others may be omitted. This diversity helps the ensemble model generalize well to unseen data.

The most well-known algorithm for bagging is Random Forest. In a random forest, a collection of decision trees is built, each trained on a different subset of data. During the training process, each tree is constructed by selecting a random subset of features for each split, adding an extra layer of randomness and diversity to the ensemble. The final prediction is then made by averaging or taking a vote among the predictions of individual trees.

One key advantage of bagging is to handle noisy data and outliers effectively. Since the ensemble model aggregates predictions from multiple base learners, the impact of outliers on overall performance is reduced. Additionally bagging is parallelizable, as each base learner can be trained independently, leading to efficient and scalable implementations

Despite its strength, bagging might not significantly improve the performance of an already stable model or one that is not prone to overfitting. It is particulary useful when dealing with complex models that have high variance, such as deep decision trees or neural networks.



→ Parallel (Bagging Concept)

Boosting : Boosting, like bagging, is an ensemble learning technique, but it aims to improve the performance of weak learners by combining them in a sequential manner. The core idea behind boosting is to give more weight to misclassified instances during the training process, enabling subsequent learners to focus on the mistakes made by their predecessors.

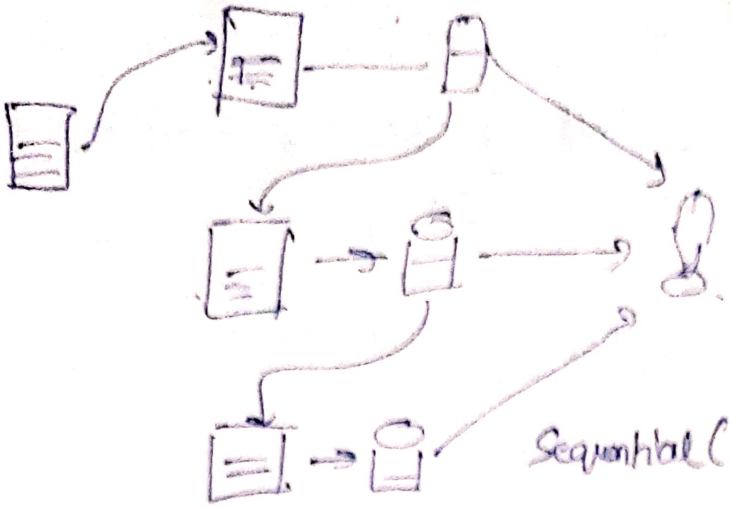
Unlike bagging, boosting does not rely on bootstrapped subset of the data. Instead, it assigns weights to each instance in the training set and adjust these weights throughout the boosting iterations. In each iteration, a new weak learner is trained on the data, and the weights of the misclassified instances are increased. This allows the subsequent learner to pay more attention to the previously misclassified samples.

The most well known boosting algorithm is AdaBoost (Adaptive Boosting). In AdaBoost, the weak learners are usually simple models with low predictive power, such as shallow decision trees or stumps. (trees with a single split). Each weak learner is trained sequentially and at each iteration, the weights of the misclassified instances are increased, forcing the model to focus on the difficult-to-classify samples.

Adaboost assigns a weight to each weak learner based on its performance, and the final prediction is made by combining the weighted predictions of all weak learners. Instances that are consistently misclassified by the ensemble receive higher weights, allowing subsequent weak learners to give more emphasis to these challenging cases.

One of the significant advantage of boosting is its ability to handle complex relationships in the data and improve the performance of weak learners significantly. Boosting often outperforms bagging when it comes to reducing both bias and variance. However boosting is more sensitive to noisy data and outliers compared to bagging.

Scanned with OKEN Scanner



Sequential (Boosting concept)

Difference between Bagging and Boosting

Bagging

1. The base learners are trained independently in parallel
2. Utilizes bootstrapping to create multiple subsets of training data
3. All base learners typically have equal weight when making final prediction
4. Robust to Noisy data and outliers due to averaging or voting mechanism, which reduces the impact of individual errors.
5. Aims to create diverse base learners. Thought the random subset of data and in case of Random forests, random feature selection for each tree.
6. Primarily reduces variance by averaging predictions from multiple models, making it effective for models with high variance.

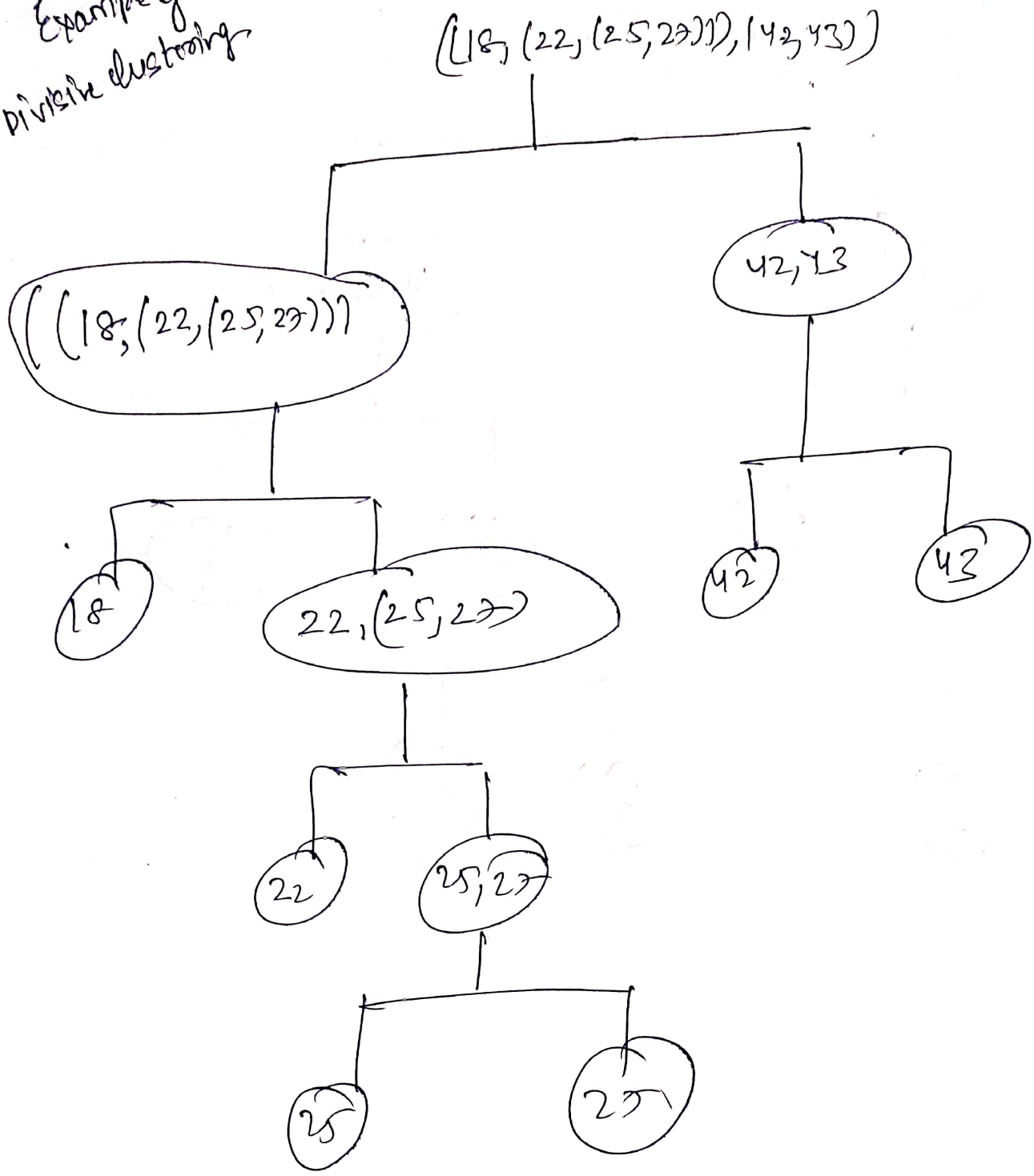
Boosting

1. The base learners are trained sequentially
2. Assign weights to instances in the training set, with higher weights given to misclassified instances to guide subsequent learners.
3. Assign different weights to each base learner based on its performance.
4. More sensitive to noisy data and outliers, as the focus on misclassified instances might lead to overfitting on these instances.
5. Focusing on improving the performance of weak learners sequentially, with each learner addressing the weakness of its predecessors.
7. Addresses both bias and variance, with a focus on reducing bias by sequentially correcting mistakes made by weak learners.

Scanned with OKEN Scanner

((18, (22, (25, 27))), (42, 43))

Example of divisive clustering



for the sequence

{18, 22, 25, 27, 42, 43}

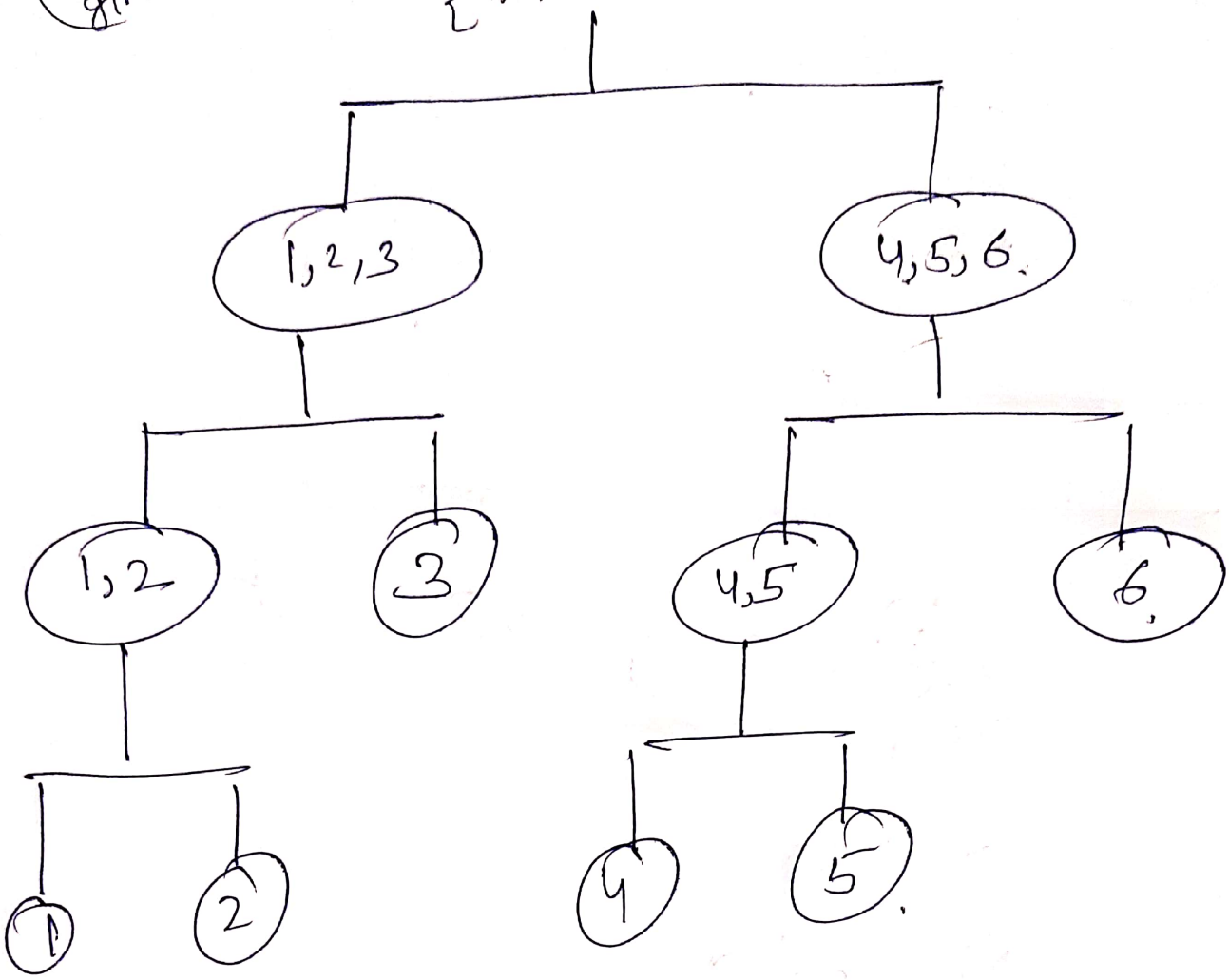
Scanned with OKEN Scanner

Eg

Apply divisive clustering for the given sequence

$\{1, 2, 3, 4, 5, 6\}$

$= \{1, 2, 3, 4, 5, 6\}$



Scanned with OKEN Scanner